

SQL и MongoDB

Синтаксис выборки, связей и изменений и правило, когда хватает таблицы, а когда документа. SQL близок к PostgreSQL. Примеры MongoDB — mongosh, те же методы, что у драйвера Node.js.

Когда что применять

SQL хранит строки в таблицах и умеет собирать их запросом. MongoDB хранит документы и удобен, когда вы читаете одну сущность целиком. Ниже — выбор модели, а не полный синтаксис.

Данные, которые читают вместе

Когда: заказ всегда показывают вместе с двумя-тремя позициями — в MongoDB это один документ. Автор и сотни книг, пользователи и роли, отчёты по разным таблицам — в SQL это отдельные таблицы и связь по ключу. Вкладывать в документ то, что живёт своей жизнью и разрастается без предела, не стоит.

SQL

```
SELECT books.title, authors.name
FROM books
JOIN authors ON authors.id = books.author_id;
```

MONGODB, ВНУТРИ ДОКУМЕНТА

```
db.posts.insertOne({
  title: "Заметка",
  comments: [
    { author: "Анна", text: "Полезно" }
  ]
})
```

MONGODB, ССЫЛКА

```
db.comments.insertOne({
  postId: postId,
  author: "Анна",
  text: "Полезно"
})
```

База должна отклонить плохую запись

Когда: у пользователя обязан быть уникальный email, а у книги — существующий автор. В SQL это NOT NULL, UNIQUE и внешний ключ. В MongoDB уникальность даёт уникальный индекс, обязательные поля — \$jsonSchema. Связи «эта книга указывает на реального автора» MongoDB сам не проверит.

SQL

```
CREATE TABLE books (  
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title text NOT NULL,  
  author_id bigint NOT NULL REFERENCES authors (id)  
);
```

УНИКАЛЬНЫЙ EMAIL

```
db.users.createIndex(  
  { email: 1 },  
  { unique: true }  
)
```

ОБЯЗАТЕЛЬНЫЕ ПОЛЯ

```
db.createCollection("users", {  
  validator: {  
    $jsonSchema: {  
      required: ["email", "name"]  
    }  
  }  
})
```

Посчитать по группам

Когда: нужен отчёт: сколько книг у автора, какая сумма оплаченных заказов. В SQL это GROUP BY. В MongoDB — конвейер aggregate. Фильтр строк ставьте до группировки: WHERE и \$match отсекают лишнее раньше.

SQL

```
SELECT author_id, COUNT(*) AS book_count
FROM books
GROUP BY author_id
HAVING COUNT(*) ≥ 2;
```

MONGODB

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: {
    _id: "$userId",
    total: { $sum: "$amount" }
  } },
  { $sort: { total: -1 } }
])
```

НЕ ПУТАТЬ

```
-- WHERE отбирает строки
-- HAVING отбирает уже посчитанные группы

// $match до $group дешевле,
// чем группировать всё и фильтровать после
```

Поменять или удалить записи

Когда: меняется одно поле — UPDATE ... SET и updateOne с \$set. Условие обязательно. UPDATE и DELETE без WHERE, как и deleteMany({}), задевают всю таблицу или коллекцию.

SQL

```
UPDATE users
SET name = 'Борис'
WHERE id = 1;
```

MONGODB

```
db.users.updateOne(
  { email: "boris@example.com" },
  { $set: { name: "Борис" } }
)
```

УДАЛЕНИЕ

```
DELETE FROM users WHERE id = 1;

db.users.deleteOne({ email: "boris@example.com" })
```

Несколько записей должны измениться вместе

Когда: списание и зачисление — одна операция: либо оба изменения, либо ни одного. В SQL это транзакция. В MongoDB, если оба поля живут в одном документе, хватает одного `updateOne`. Транзакция на несколько документов нужна, только когда их нельзя честно сложить в один документ.

SQL

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

ОДИН ДОКУМЕНТ

```
db.accounts.updateOne(  
  { _id: id, balance: { $gte: 100 } },  
  { $inc: { balance: -100 } }  
)
```

НЕСКОЛЬКО ДОКУМЕНТОВ

```
session.startTransaction()  
orders.updateOne(  
  { _id: id },  
  { $set: { status: "paid" } },  
  { session }  
)  
session.commitTransaction()
```

Дальняя страница длинного списка

Когда: первая страница короткая — хватает LIMIT и limit. OFFSET и skip на десятой тысяче заставляют базу прочитать и выбросить все предыдущие строки. Для глубокой листалки берите «после такого-то id».

SQL, ПЕРВАЯ СТРАНИЦА

```
SELECT id, title
FROM books
ORDER BY id
LIMIT 20;
```

SQL, СЛЕДУЮЩАЯ

```
SELECT id, title
FROM books
WHERE id > 100
ORDER BY id
LIMIT 20;
```

MONGODB

```
db.books.find({ _id: { $gt: lastId } })
  .sort({ _id: 1 })
  .limit(20)
```

SQL

Запрос говорит, какие строки нужны, а не как их обходить циклом. Имена таблиц ниже — authors, books, users, accounts.

SELECT и WHERE

выборка

Когда: перечисляйте нужные столбцы. SELECT * удобен в консоли, в коде приложения он тащит лишние поля и ломается, если в таблице появился тяжёлый столбец. WHERE отбирает строки до группировки.

```
SELECT title, year
FROM books
WHERE year BETWEEN 2020 AND 2026
      AND status IN ('draft', 'published')
ORDER BY year DESC;
```

LIKE 'SQL%' ищет строки с таким началом и может использовать индекс. LIKE '%SQL' — нет: процент в начале заставляет смотреть все строки.

NULL

пусто

Когда: значения нет. NULL не равен ничему, в том числе другому NULL. Сравнение = NULL не находит строки. Нужны IS NULL и IS NOT NULL.

```
SELECT id, email
FROM users
WHERE email IS NULL;
```

JOIN

связь

Когда: JOIN (он же INNER JOIN) оставляет только строки, у которых нашлась пара. LEFT JOIN оставляет все строки слева, даже если справа пары нет: недостающие столбцы будут NULL. Берите LEFT JOIN, когда отсутствие связи — нормальный ответ, а не повод выбросить книгу из списка.

```
SELECT books.title, authors.name
FROM books
LEFT JOIN authors ON authors.id = books.author_id
WHERE books.year ≥ 2020
ORDER BY books.title;
```

Если нужны только столбцы одной таблицы, а вторая лишь проверяет условие, подойдёт и WHERE author_id IN (SELECT id FROM authors WHERE country = 'IL'). Как только из второй таблицы нужны свои столбцы, JOIN читается прямее.

GROUP BY и HAVING

отчёт

Когда: одна строка ответа — это группа, а не исходная запись. В списке SELECT рядом с агрегатом только столбцы из GROUP BY. WHERE отбрасывает строки заранее, HAVING отбрасывает уже посчитанные группы.

```
SELECT author_id, COUNT(*) AS book_count
FROM books
WHERE year ≥ 2020
GROUP BY author_id
HAVING COUNT(*) ≥ 2
ORDER BY book_count DESC;
```

COUNT(*) считает строки. COUNT(email) считает непустые email. COUNT(DISTINCT country) считает разные значения, а не просто включает DISTINCT в список.

INSERT

запись

Когда: появляется новая строка. Столбцы перечисляйте явно, чтобы порядок значений не зависел от порядка колонок в таблице.

```
INSERT INTO users (email, name)
VALUES ('boris@example.com', 'Борис');
```

UPDATE и DELETE

изменение

Когда: меняется или исчезает уже существующая строка. Сначала напишите SELECT с тем же WHERE и посмотрите, сколько строк он находит. Без WHERE команда применяется ко всей таблице.

```
UPDATE books
SET title = 'Новое название', year = 2026
WHERE id = 15;

DELETE FROM books
WHERE id = 15;
```

Таблица, ключ и связь

[схема](#)

Когда: появляется новый вид сущностей. Первичный ключ однозначно называет строку. Внешний ключ не даёт вставить книгу с автором, которого нет, и удерживает от удаления автора, на которого ещё есть книги.

```
CREATE TABLE authors (  
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name text NOT NULL  
);  
  
CREATE TABLE books (  
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title text NOT NULL,  
  author_id bigint NOT NULL REFERENCES authors (id)  
);
```

Индекс

[поиск](#)

Когда: по столбцу часто фильтруют, соединяют таблицы или сортируют. Первичный ключ и UNIQUE уже создают индекс. Индекс на каждый столбец замедляет INSERT и UPDATE: каждое изменение надо записать и в индекс.

```
CREATE UNIQUE INDEX users_email_idx ON users (email);  
  
CREATE INDEX books_author_id_idx ON books (author_id);
```

Транзакция

атомарность

Когда: несколько команд обязаны выполняться вместе. COMMIT сохраняет их. ROLLBACK отменяет всё с BEGIN, если вторая команда не удалась. Для одного UPDATE отдельная транзакция в прикладном коде обычно не нужна: одна команда и так атомарна.

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

MongoDB

Документ — это объект, коллекция — набор таких объектов. Поле `_id` база ставит сама, если его не передать. Примеры ниже рассчитаны на базу, в которой уже выполнено `use shop`.

insertOne и insertMany

запись

Когда: появляется один документ или пачка независимых. Поля у документов в одной коллекции могут различаться. Это удобно для черновика и опасно для данных, по которым вы потом строите отчёт: отсутствующее поле легко забыть в запросе.

```
db.books.insertOne({  
  title: "SQL",  
  year: 2024,  
  tags: ["sql"]  
})  
  
db.books.insertMany([  
  { title: "MongoDB", year: 2025 },  
  { title: "Индексы", year: 2026, tags: ["sql"] }  
])
```

find, фильтр и проекция

выборка

Когда: первый аргумент — какие документы нужны, второй — какие поля вернуть. 1 оставляет поле, 0 убирает. `_id` возвращается сам, пока его явно не выключить. Пустой фильтр `{}` — это вся коллекция.

```
db.books.find(  
  { year: { $gte: 2020 }, tags: "sql" },  
  { title: 1, year: 1, _id: 0 }  
)
```

`tags: "sql"` находит документ, в массиве `tags` которого есть `"sql"`. Отдельный оператор для «элемент массива» здесь не нужен.

Сравнения и отсутствующее поле

фильтр

Когда: условие сложнее равенства. Нет поля и поле равно `null` — разные ситуации. `$exists: false` ловит документы, где ключа нет вовсе.

```
db.books.find(  
  status: { $in: ["draft", "published"] },  
  year: { $gte: 2020, $lt: 2027 }  
)  
  
db.users.find({ email: { $exists: false } })
```

updateOne и \$set

изменение

Когда: нужно поменять несколько полей, не затерев остальные. \$set пишет только перечисленные ключи. replaceOne подменяет документ целиком. updateMany — когда условие должно задеть все подходящие документы, а не первый.

```
db.books.updateOne(
  { _id: id },
  { $set: { title: "Новое название", year: 2026 } }
)

db.books.replaceOne(
  { _id: id },
  { title: "Только эти поля", year: 2026 }
)
```

\$inc увеличивает число. Условие { balance: { \$gte: 100 } } в том же фильтре не даёт уйти в минус, если параллельный запрос уже списал деньги.

deleteOne и deleteMany

удаление

Когда: документ больше не нужен. deleteOne снимает первый подходящий. deleteMany требует осознанный фильтр. Пустой объект удалит все документы коллекции, саму коллекцию при этом не уничтожив.

```
db.books.deleteOne({ _id: id })

db.books.deleteMany({ status: "draft" })
```

Когда: комментариев мало и их всегда показывают вместе с заметкой — держите массив внутри документа. Комментариев много, их листают отдельно или один и тот же автор нужен в разных местах — храните отдельную коллекцию и поле `postId`. Документ в MongoDB ограничен 16 МБ, массив «всё внутри» упирается в этот потолок.

```
db.posts.insertOne({
  title: "Заметка",
  comments: [{ author: "Анна", text: "Полезно" }]
})

db.comments.insertOne({
  postId: postId,
  author: "Анна",
  text: "Полезно"
})
```

aggregate

конвейер

Когда: простого `find` мало: нужна группировка или сборка из другой коллекции. Стадии идут по порядку. `$lookup` похож на левое соединение, но тяжелее обычного `find` по вложенным полям. Если автор нужен при каждом чтении книги, имя автора проще хранить в самой книге.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])

db.books.aggregate([
  { $match: { year: { $gte: 2020 } } },
  { $lookup: {
    from: "authors",
    localField: "authorId",
    foreignField: "_id",
    as: "author"
  } }
])
```

Индекс

поиск

Когда: по полю ищут, сортируют или требуют уникальность. `_id` уже проиндексирован. Уникальный индекс не заставляет поле быть в каждом документе: два документа без `email` для простого уникального индекса допустимы. Обязательность поля задают валидатором, а не индексом.

```
db.users.createIndex({ email: 1 }, { unique: true })

db.books.createIndex({ authorId: 1, year: -1 })
```

Порядок полей в составном индексе важен. Индекс `{ authorId: 1, year: -1 }` помогает запросу по автору и запросу по автору с сортировкой по году. Запросу «только по году» он не замена отдельного индекса по `year`.

Когда: нужен порядок и не вся коллекция. `sort` без индекса на большой коллекции дорогой. `skip` для дальних страниц читает и выбрасывает пропущенные документы. Следующую страницу надёжнее начинать с последнего увиденного `_id`.

```
db.books.find({ year: { $gte: 2020 } })  
  .sort({ title: 1 })  
  .limit(20)
```

```
db.books.find({ _id: { $gt: lastId } })  
  .sort({ _id: 1 })  
  .limit(20)
```